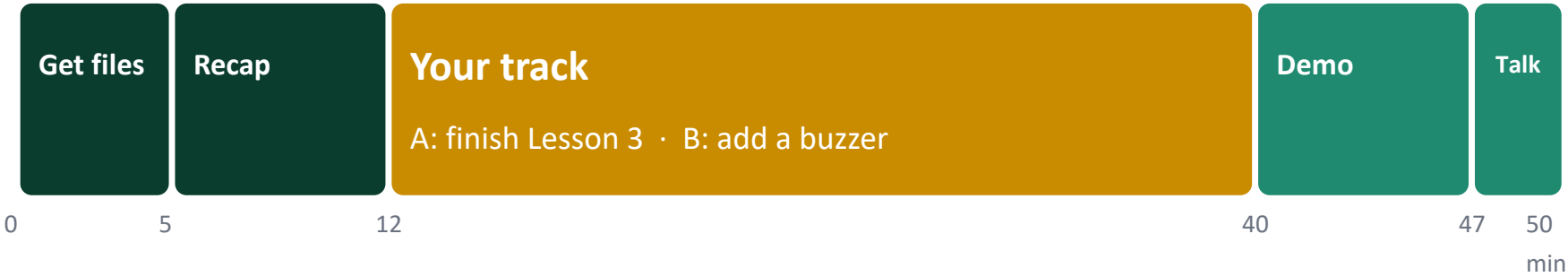
Beep!

Lesson 4 · Add sound to the distance meter

JA05 STEAM and Service Learning · 16 Oct 2026



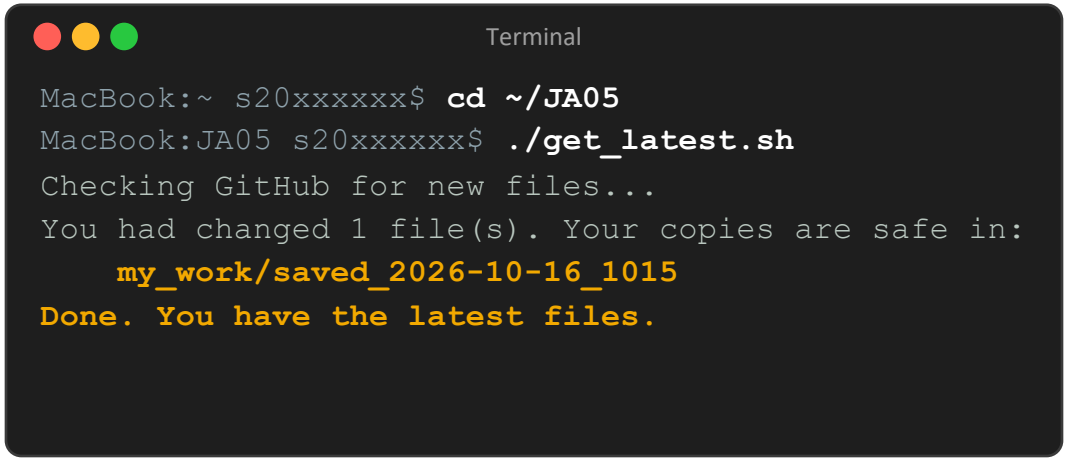
Today: 50 minutes



Talk = who needs light, and who needs sound?

Every group leaves with something that works. It does not matter which track.

New: one command gets everything



1. Saves files you changed by mistake into my_work
2. Gets the newest files from GitHub
3. Never touches your ESP32

CHECKPOINT You see: Done. You have the latest files.

STUCK? Not found? Do the next slide first.

First time today

Your JA05 does not have get_latest.sh yet. Get it with git pull:

Terminal

```
MacBook:~ s20xxxxxx$ cd ~/JA05
MacBook:JA05 s20xxxxxx$ git pull
```

It worked

You now have get_latest.sh. Use it from next time.

An error?

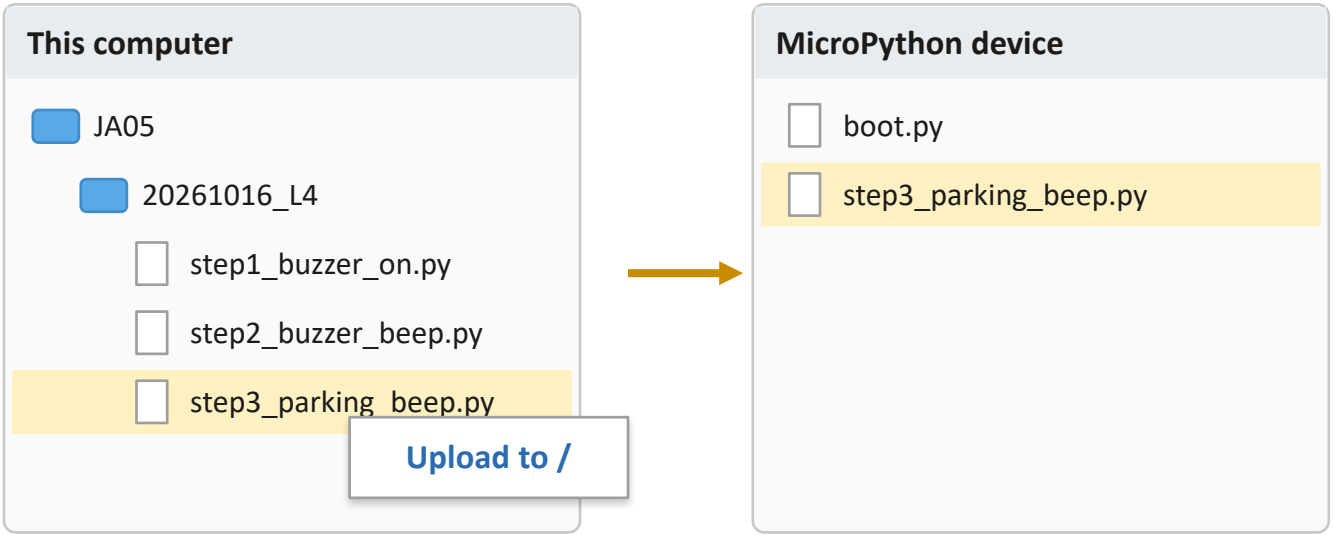
For example: local changes would be overwritten

Terminal

```
MacBook:~ s20xxxxxx$ cd ~
MacBook:~ s20xxxxxx$ mv JA05 JA05_old
MacBook:~ s20xxxxxx$ git clone
https://github.com/jackyhui/JA05.git
```

Your old files stay in JA05_old.

Your code lives on your ESP32



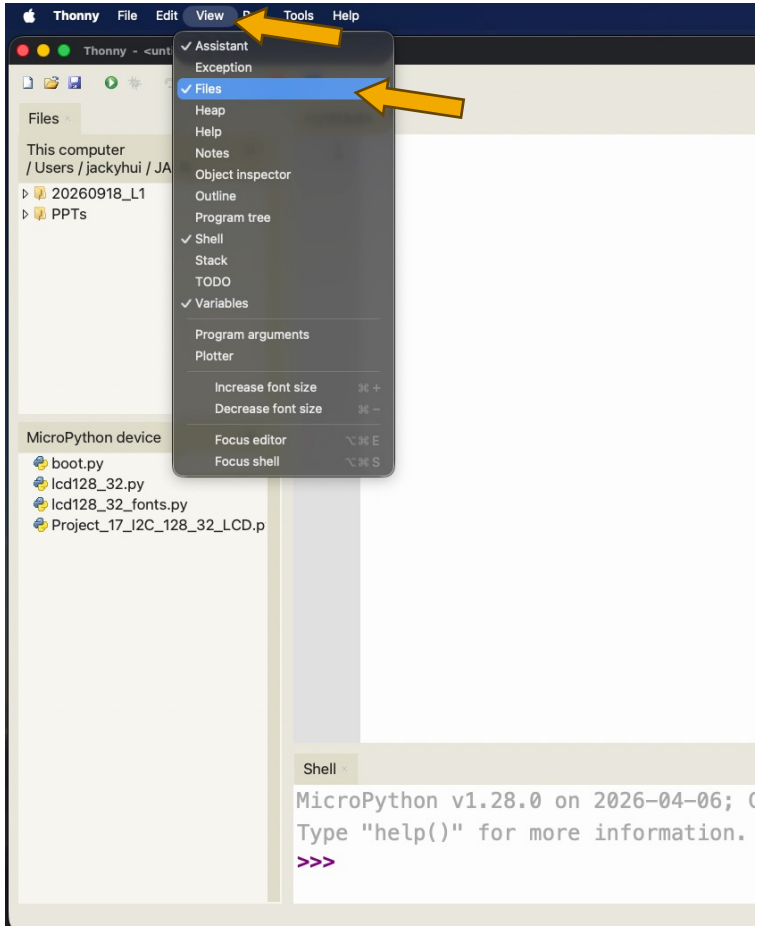
View > Files in Thonny

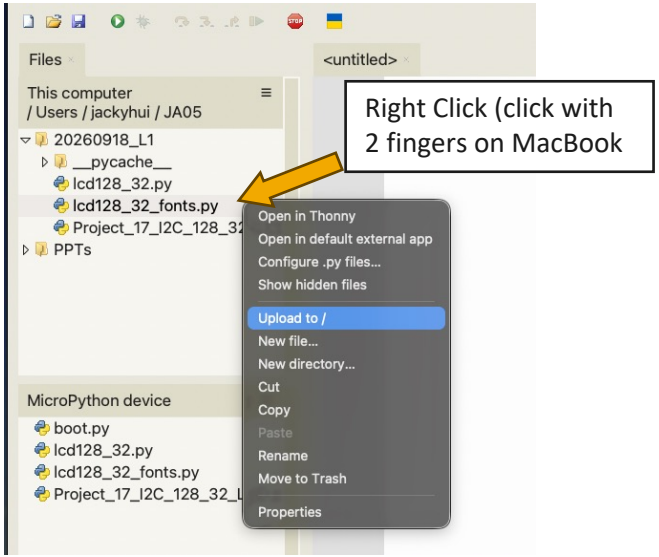
Edit HERE

JA05 on your computer = the teacher's copy. Never edit it.

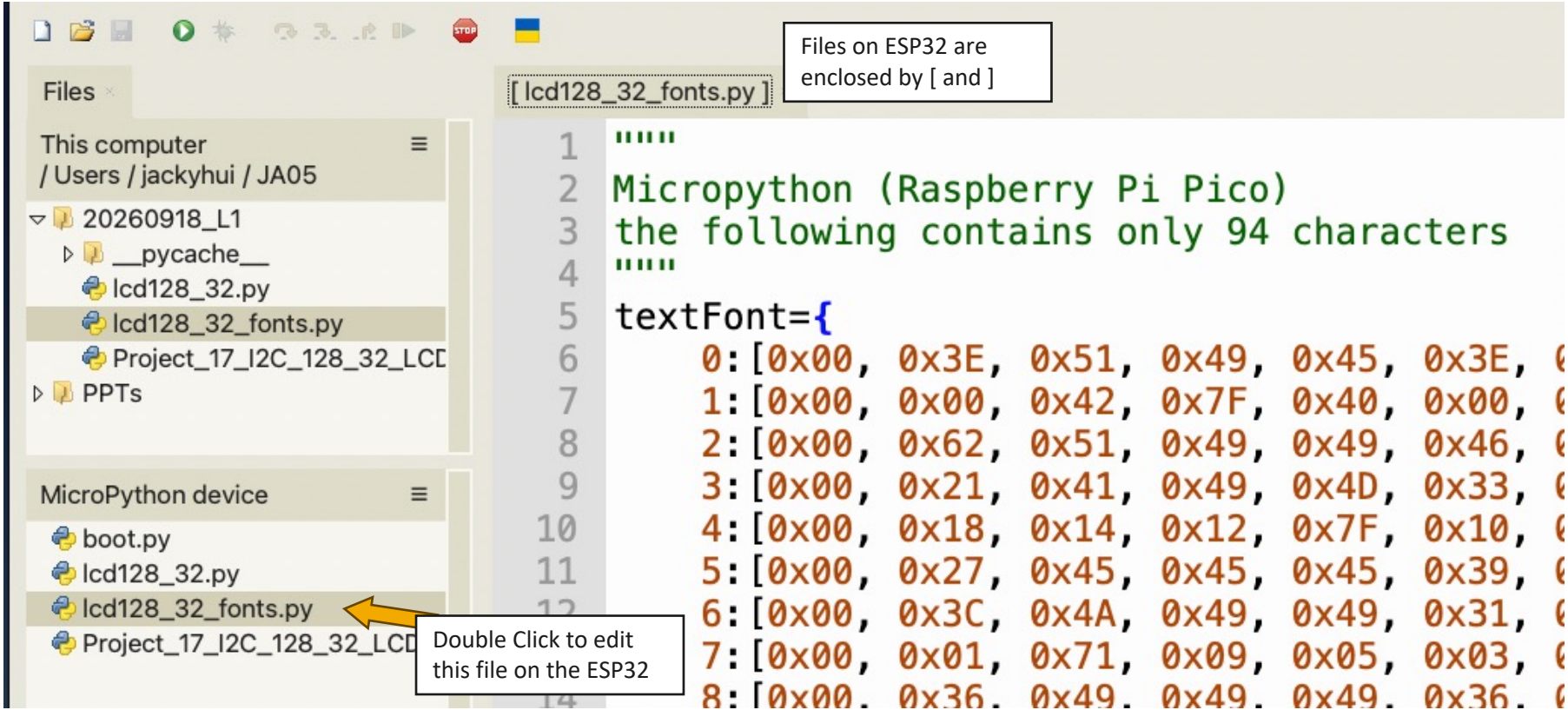
Your copy is on the ESP32. get_latest.sh never touches it.

CHECKPOINT You edit files under MicroPython device only.





Right Click (click with 2 fingers on MacBook


Files
This computer
/ Users / jackyhui / JA05
20260918_L1
__pycache__
lcd128_32.py
lcd128_32_fonts.py
Project_17_I2C_128_32_LCD
PPTs
MicroPython device
boot.py
lcd128_32.py
lcd128_32_fonts.py
Project_17_I2C_128_32_LCD
Double Click to edit this file on the ESP32
[lcd128_32_fonts.py]
Files on ESP32 are enclosed by [and]
"""
Micropython (Raspberry Pi Pico)
the following contains only 94 characters
"""
textFont={
0:[0x00, 0x3E, 0x51, 0x49, 0x45, 0x3E,
1:[0x00, 0x00, 0x42, 0x7F, 0x40, 0x00,
2:[0x00, 0x62, 0x51, 0x49, 0x49, 0x46,
3:[0x00, 0x21, 0x41, 0x49, 0x4D, 0x33,
4:[0x00, 0x18, 0x14, 0x12, 0x7F, 0x10,
5:[0x00, 0x27, 0x45, 0x45, 0x45, 0x39,
6:[0x00, 0x3C, 0x4A, 0x49, 0x49, 0x31,
7:[0x00, 0x01, 0x71, 0x09, 0x05, 0x03,
8:[0x00, 0x36, 0x49, 0x49, 0x49, 0x36,

Lesson 3 answer: leds_for()

step5_solution.py · lines 48–53

```
48  def leds_for(distance):
  ⋮
50      if distance is None:
51          return 0
52      count = 5 - int(distance // STEP_CM)
53      return max(0, min(5, count))
```

With maths: 5 minus the number of 5 cm steps, kept between 0 and 5.

CHECKPOINT Test: 12 cm should give 3 LEDs.

Same idea with if / elif

```
if distance < 5:
    return 5
elif distance < 10:
    return 4
...
else:
    return 0
```

Both give the same answer. Yours can look different and still be right.

Pick your track

Does your LED bar work? (Lesson 3, Step 5)

A — Not yet

- Finish the distance meter
- Use the solution file
- Then join Track B

B — Yes, it works

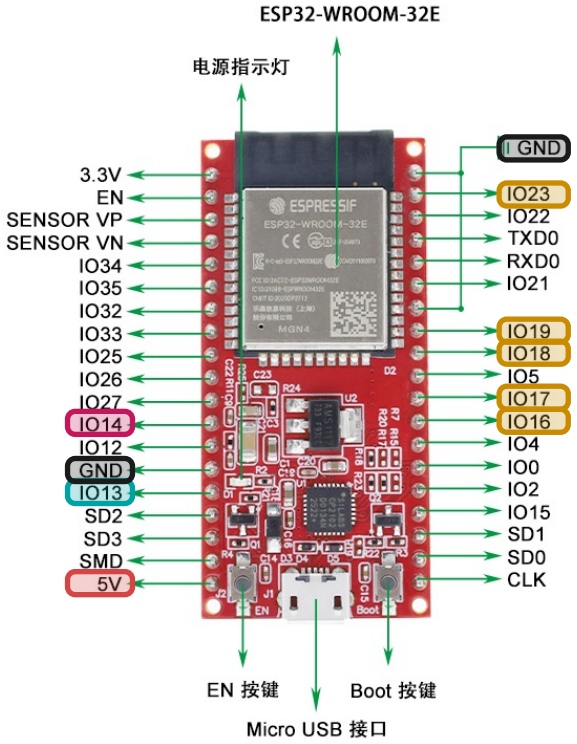
- Add a buzzer
- Beep faster when closer
- Stretch: change the pitch

Track A: finish the distance meter

A

1. Upload the Lesson 3 files you need to the ESP32
2. Check the LEDs: step3a_check_leds.py
3. Add the sensor: Lesson 3 PDF, page 25
4. Run step4_ultrasonic_print.py, then step5_solution.py
5. LED bar works? Go to Track B

CHECKPOINT Hand closer, more LEDs on.



Two buzzers in your kit

Active buzzer

makes its own tone



underneath

Sealed black back

+ to 3V3: steady beep

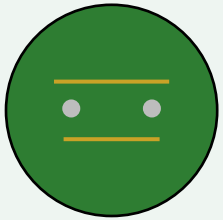
Code: value(1)

Use for Steps 0–3

有源蜂鸣器 · "with power"

Passive buzzer

needs the ESP32 to make the tone



underneath

Green circuit board

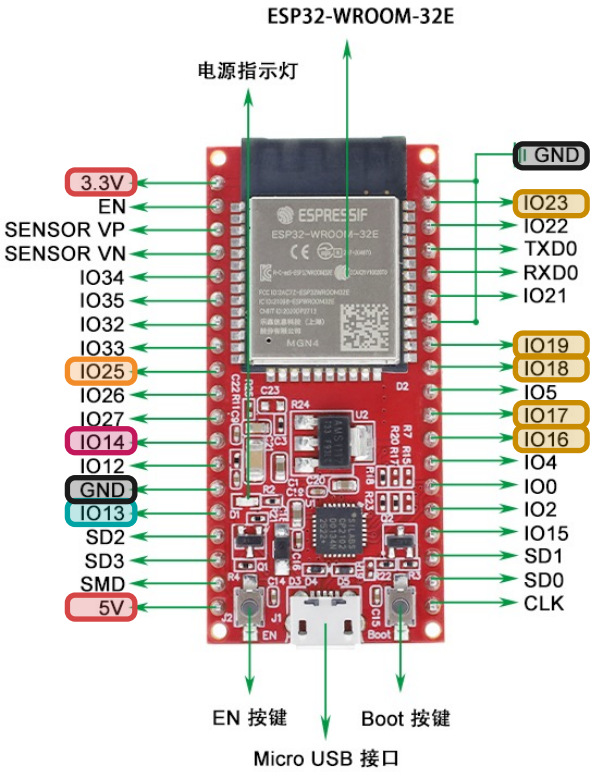
+ to 3V3: one click only

Code: PWM, choose the note

Use for the Step 4 stretch

无源蜂鸣器 · "without power"

Find your pins



On the board	Used for
IO25	**Buzzer + (new today)**
3.3V	Buzzer in Step 0 only
IO16, 17, 18, 19, 23	LED 1 – 5
5V	Sensor VCC
IO13 / IO14	Sensor Trig / Echo
GND	Every minus leg

The board says IO25. The code says Pin(25). Same pin.

There are three GND pins. Any of them works.

How to use the code today

1. Bottom-right of Thonny says **MicroPython (ESP32)**
2. **View > Files**, then open JA05 > 20261016_L4
3. Right-click the step file > **Upload to /**
4. Double-click it under **MicroPython device**
5. **Run ▶️ (F5)** without changes first
6. Change only Try it / YOUR CODE lines. **⌘S** saves on the ESP32

Don't type code from the slides

Slides show only PART of a program. The full program is already in the file.

step2_buzzer_beep.py · lines 13–18

This header tells you which file and which lines to look at.

Step 0: beep with no code

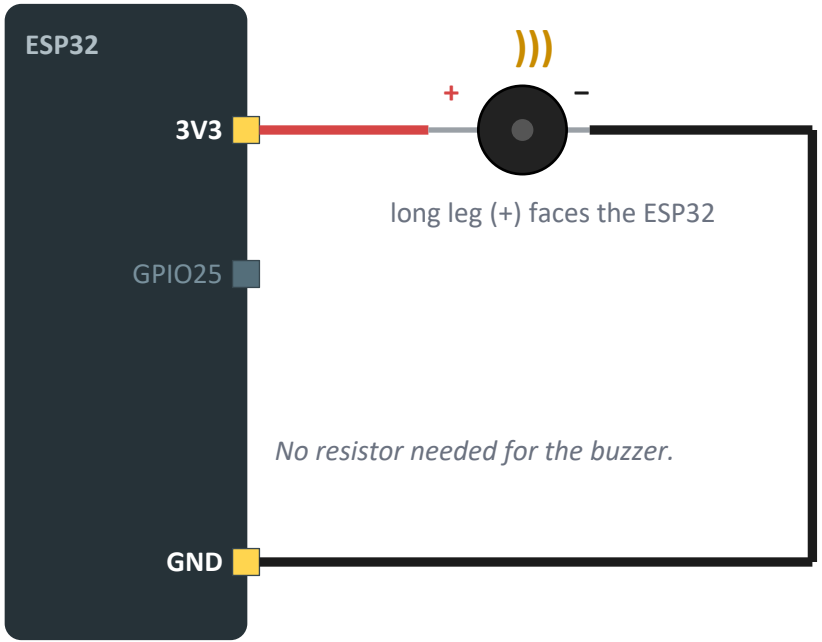
0

1. Keep the Lesson 3 circuit plugged in.
2. Take the ACTIVE buzzer.
3. Red wire: 3V3 → buzzer + (long leg).
4. Black wire: buzzer − → GND rail.

CHECKPOINT Steady beep as soon as USB is in.

STUCK? Only a click? Wrong buzzer. Swap it.

3-pin board? S → GPIO25, V → 3V3, G → GND



Step 1: let the ESP32 beep

1

Move ONE wire: from 3V3 to GPIO25 (IO25 on the board).

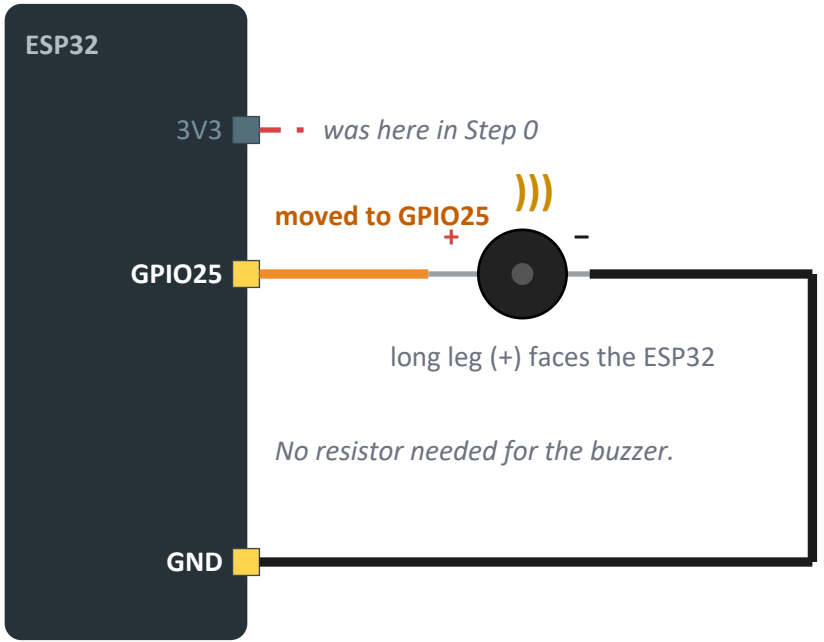
step1_buzzer_on.py · lines 12–16

```
12  from machine import Pin
13
14  buzzer = Pin(25, Pin.OUT)
15
16  buzzer.value(1)           # 1 = beep
```

Run the whole file. Then try: change 1 to 0 on line 16.

CHECKPOINT Beeps after you run the code.

STUCK? Wire back to 3V3. Does Step 0 still beep?



Read the code: Step 1

1

step1_buzzer_on.py · lines 12–17

```
12  from machine import Pin
13
14  buzzer = Pin(25, Pin.OUT)
15
16  buzzer.value(1)
17  print("Buzzer should be beeping")
```

Line 12 Bring in Pin, the tool that controls the pins.

Line 14 GPIO25 is an OUTPUT: the ESP32 sends power out. We name it buzzer.

Line 16 value(1) puts 3.3 V on the pin, so it beeps. value(0) stops it.

Line 17 Prints a message in the Shell, so you know the code ran.

Predict, then try

- What happens if line 16 says value(0)?
- Where have you seen Pin(…, Pin.OUT) before?

Step 2: beep, beep, beep

2

No wiring change. Upload and run step2_buzzer_beep.py.

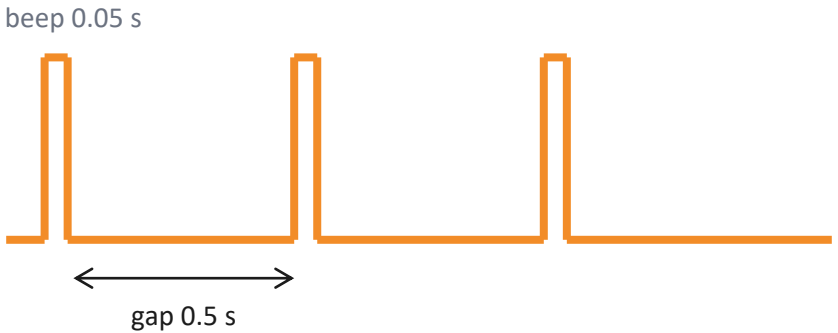
step2_buzzer_beep.py · lines 13–18

```
13  try:
14      while True:
15          buzzer.value(1)     # beep
16          time.sleep(0.05)    # a short beep: 0.05 s
17          buzzer.value(0)     # quiet
18          time.sleep(0.5)     # wait 0.5 s
```

Ctrl+C to stop · Try it: line 18, change 0.5 to 0.1

CHECKPOINT Short beeps, about twice a second.

What the pin does



Line 16 sets the beep. Line 18 sets the gap.

Read the code: Step 2

2

step2_buzzer_beep.py · lines 13–22

```
13  try:
14      while True:
15          buzzer.value(1)
16          time.sleep(0.05)
17          buzzer.value(0)
18          time.sleep(0.5)
19
20          # Try it: change 0.5 to 0.1
21  finally:
22      buzzer.value(0)
```

Line 14 while True: repeat the indented lines forever.

Line 15–16 Beep, and keep beeping for 0.05 s.

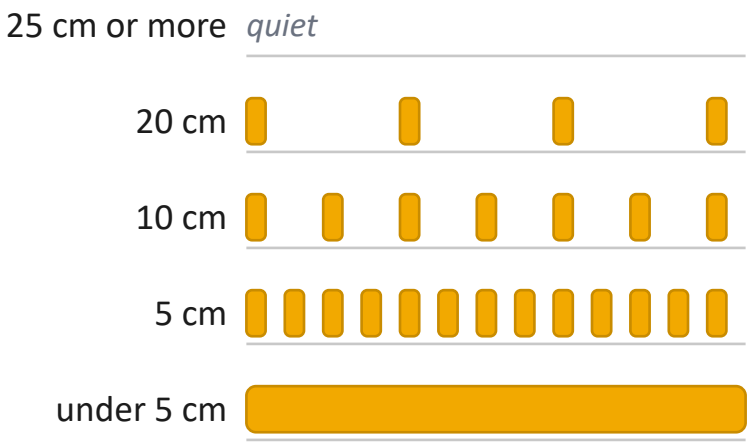
Line 17–18 Quiet, and wait 0.5 s. Then back to line 15.

Line 13, 21 try / finally: when you press Ctrl+C, line 22 runs and the buzzer stops.

Predict, then try: swap 0.05 and 0.5. What do you hear? Why?

Step 3: closer = faster beeps

3



The gap between beeps gets shorter as you get closer.

STUCK? Always the same speed? Is line 65 still 0.5?

step3_parking_beep.py · lines 54–66

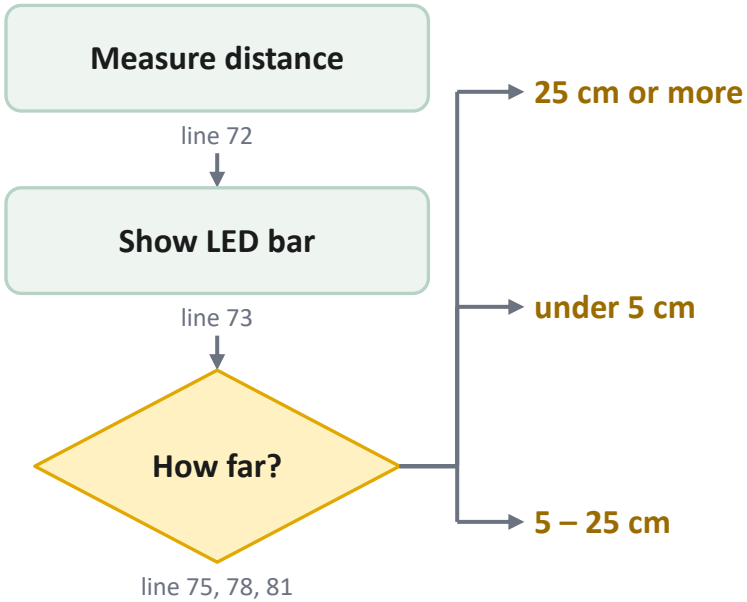
```
54  def gap_for(distance):
 ⋮
58      # ----- YOUR CODE STARTS HERE -----
 ⋮
65      return 0.5
66      # ----- YOUR CODE ENDS HERE -----
```

Your job: replace line 65 so the gap depends on distance. Do not change any other lines.

CHECKPOINT Beeps speed up as your hand comes closer.

Read the code: the main loop

3



Quiet

lines 76–77

Long beep

lines 79–80

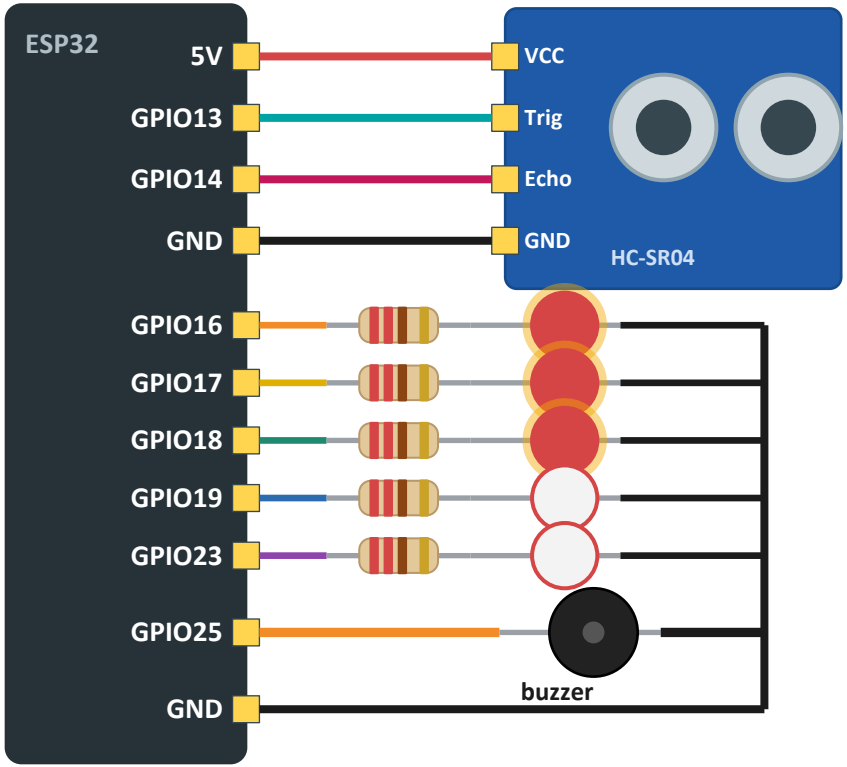
Short beep, then wait
gap_for(distance)

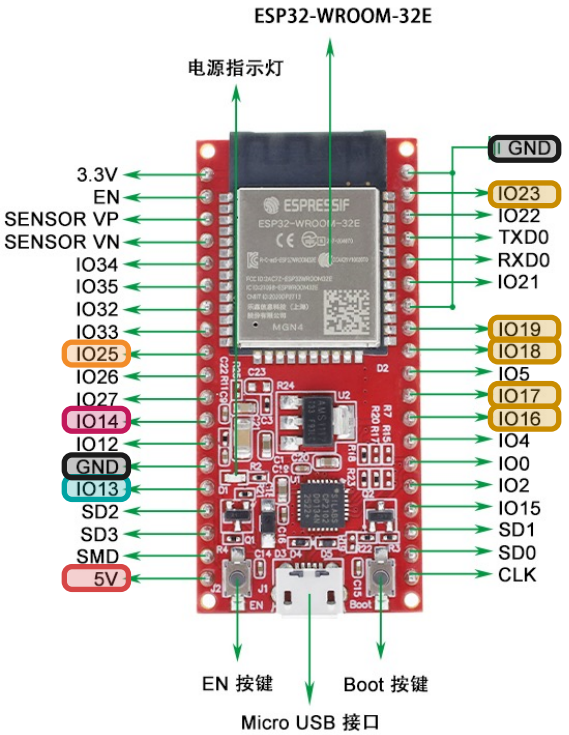
lines 82–85

Then back to the top. Again and again, 10 times a second or faster.

Your gap_for() is used only here

The whole circuit





Outlined: pins used today. Orange = buzzer.

Stretch: change the note

4

Swap to the PASSIVE buzzer. Same two wires. Upload and run step4_stretch_pitch.py.

step4_stretch_pitch.py · lines 22–64

```
22  buzzer = PWM(Pin(25), freq=1000, duty=0)
  ⋮
60  if distance is None or distance >= 25:
61      buzzer.duty(0)
62  else:
63      buzzer.freq(pitch_for(distance))
64      buzzer.duty(512)
```

CHECKPOINT The note gets higher as your hand comes closer.

Try it: change pitch_for() so the note goes DOWN as you get closer.

far away: 300 Hz, low note



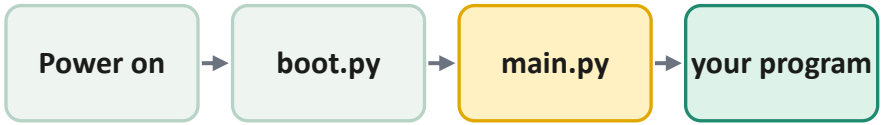
very close: 2300 Hz, high note



Line 22: PWM switches the pin on and off very fast. Line 63: freq sets how fast. Line 64: duty 512 = sound on.

Run by itself when powered on

When the ESP32 starts, it runs only two files, in this order:



step3_parking_beep.py is not main.py, so it only runs when you press Run.

1. Your step3_parking_beep.py is on the ESP32
2. Upload 20261016_L4/main.py too
3. Unplug, plug in again: it starts by itself

Now it works on a USB power bank, away from the computer.

main.py · line 10

```
10  import step3_parking_beep
```

main.py starts your program by its name (no .py). To run a different one, change this line.

Thonny cannot connect? Click Stop ■ to interrupt it.

Turn it off: right-click main.py under MicroPython device > Delete.

Light, sound, or both?



Low vision

Cannot see the LEDs. Needs sound.



Hard of hearing

Cannot hear the beep. Needs light.



Noisy place

A busy road or kitchen. Light works better.

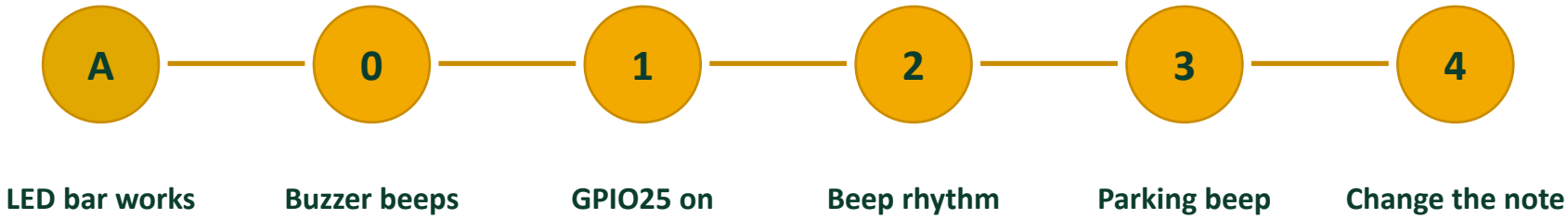


Everyone

Give both. Then more people can use it.

Who in our community could use your distance meter? Light, sound, or both?

What did you reach today?



- An active buzzer is an output, like an LED.
- Edit on the ESP32, never in JA05. Start every lesson with ./get_latest.sh.
- Light and sound together help more people.